Method 3: Parallel Multi-Process EEVEE Pipelining

Overlapping CPU Scene Evaluation, GPU Shading & NVMe SSD Lock-File Coordination Across 2 Parallel Workers

Tested on: Blender 4.4.3 · Windows · Intel Core i7-1255U (10 cores / 12 threads) · Intel Iris Xe · 16 GB RAM · NVMe SSD. Utilization and timing figures in this guide come from that single machine; yours will differ.
Blender 5.x: the EEVEE engine id is `BLENDER_EEVEE_NEXT` in Blender 4.2–4.5 and `BLENDER_EEVEE` in 5.x. The scripts detect the right id automatically; every other property was verified on Blender 4.4.3 only.

1. About This Method, Hardware Profile & When It Is Preferred

How the Method Works: Every single frame in Blender goes through three sequential stages: (1) CPU dependency-graph and mesh evaluation, (2) GPU pixel shading (EEVEE Next), and (3) SSD file writing. In a single window, the GPU sits idle while the CPU prepares the frame and saves the PNG, and the CPU sits idle while the GPU shades. Method 3 launches two simultaneous headless EEVEE processes (4 CPU threads each = 8 threads total) pointing to the same folder with `use_placeholder = True` and `use_overwrite = False`. While Worker 1 shades frame N on the GPU, Worker 2 evaluates frame N+1 on the CPU and writes to the NVMe SSD — eliminating idle gaps while keeping 100% identical EEVEE lighting across every frame.

Hardware Component	Target Utilization	Exact Role in Method 3
GPU (Graphics Card / iGPU)	100% Continuous	Alternates shading between Worker 1 and Worker 2 with zero idle gaps between frames.
CPU (Processor Cores)	35% – 55% (4 + 4 = 8 Threads)	Worker 2 evaluates animations/modifiers on 4 threads while Worker 1 finishes GPU shading.
System RAM (Memory)	Moderate (~4.4 – 6.0 GB Total)	Holds two independent ~2.2 GB headless scene instances in RAM (`use_persistent_data = True`).
NVMe SSD Storage	Real-Time Traffic Controller	Coordinates workers via 0-byte placeholder lock files (`frame_xxxx.png`) and writes raw PNGs in 0.04s.

When Is Method 3 Preferred?

- **16 GB+ RAM Laptops & Desktops with Fast SSDs:** When a single headless EEVEE instance uses only 2–3 GB of RAM, leaving plenty of memory to run 2 parallel instances (~4.5 GB total) while maintaining 100% identical EEVEE visuals.
- **Animation-Heavy or Modifier-Heavy Scenes:** When armatures, cloth/physics caches, or geometry nodes cause a 1–3 second CPU preparation pause before the GPU starts shading each frame.

2. Complete Step-by-Step Guide (From Setup to Final MP4 Video)

Step 1: Unlock OS Power Plan

```
# WINDOWS (Command Prompt as Admin):
powercfg /setactive 8c5e7fda-e8bf-4a96-9a85-a6e23a8c635c

# LINUX (Bash Terminal):
sudo cpupower frequency-set -g performance
```

Step 2: Save the Shared Multi-Process Script (`method3_parallel_eevee.py`)

Save the script below as `C:\Temp_Render\method3_parallel_eevee.py` (Windows) or `~/Temp_Render/method3_parallel_eevee.py` (Linux/macOS). Both workers run this **same** file. It is also in the repo under `scripts/`.

```
# Method 3 - Parallel Multi-Process EEVEE Pipelining   [scripts/method3_parallel_eevee.py]
# Tested on Blender 4.4.3. Launch TWO processes with this same file (-s 1 and -s 2), e.g.:
#   blender -b --factory-startup SCENE.blend -P THIS_FILE -t 4 -s 1 -e 4440 -j 1 -a
#   blender -b --factory-startup SCENE.blend -P THIS_FILE -t 4 -s 2 -e 4440 -j 1 -a
import bpy, os, platform

# ============================================================================
# USER CUSTOMIZATION SETTINGS
# ============================================================================
# Each worker uses 4 threads (2 workers x 4 threads = 8 CPU threads total!)
THREADS_PER_WORKER = 4
FRAME_STEP = 1         # 1 = Render EVERY frame sequentially (1, 2, 3, 4...)
KEEP_IN_RAM = True     # Set True on 16GB+ RAM PCs; False on 8GB RAM PCs
ENABLE_SHADOWS = False # False fixes "Shadow buffer full" on integrated GPUs
SAMPLES = 4            # 4-8 for fast animation; 16-32 for dedicated GPUs
RES_PERCENT = 50       # 50 = 960x540; 100 = full 1920x1080

# ============================================================================
# ENGINE & SSD PLACEHOLDER LOCK CONFIGURATION
# ============================================================================
scene = bpy.context.scene
render = scene.render
eevee = scene.eevee

# EEVEE engine id: 'BLENDER_EEVEE_NEXT' on Blender 4.2-4.5, 'BLENDER_EEVEE' on Blender 5.x
try:
    render.engine = 'BLENDER_EEVEE_NEXT'
except TypeError:
    render.engine = 'BLENDER_EEVEE'
scene.frame_step = FRAME_STEP
render.threads_mode = 'FIXED'
render.threads = THREADS_PER_WORKER
render.use_compositing = False
render.use_sequencer = False
render.use_persistent_data = KEEP_IN_RAM

eevee.taa_render_samples = SAMPLES
eevee.use_raytracing = False
eevee.use_shadows = ENABLE_SHADOWS
eevee.shadow_step_count = 2
eevee.use_volumetric_shadows = False
render.use_motion_blur = False

render.use_simplify = True
render.simplify_subdivision_render = 0
render.simplify_child_particles_render = 0.0
render.resolution_x = 1920
render.resolution_y = 1080
render.resolution_percentage = RES_PERCENT

out = "C:/Temp_Render/" if platform.system() == "Windows" else os.path.expanduser("~/Temp_Render/")
os.makedirs(out, exist_ok=True)
render.filepath = os.path.join(out, "frame_")

# CRITICAL FOR MULTI-PROCESS: Prevents both workers from rendering the same frame!
render.use_overwrite = False
render.use_placeholder = True
render.image_settings.file_format = 'PNG'
render.image_settings.color_mode = 'RGB'
render.image_settings.color_depth = '8'
render.image_settings.compression = 0

print(">>> METHOD 3 (PARALLEL EEVEE WORKER) READY <<<")
```

Step 3: Launch Both Parallel Workers Simultaneously (All OS)

Worker 1 starts at `-s 1` and Worker 2 starts at `-s 2`, so they never collide on the very first millisecond.

```
# WINDOWS: open TWO Command Prompt windows and paste one command into each.
# Window 1 (4 Threads + GPU + SSD):
"C:\Program Files\Blender Foundation\Blender 4.4\blender.exe" -b --factory-startup ^
  "C:\Path\To\Your_Scene.blend" ^
  -P "C:\Temp_Render\method3_parallel_eevee.py" -t 4 -s 1 -e 4440 -j 1 -a

# Window 2 (4 Threads + GPU + SSD = 8 threads total):
"C:\Program Files\Blender Foundation\Blender 4.4\blender.exe" -b --factory-startup ^
  "C:\Path\To\Your_Scene.blend" ^
  -P "C:\Temp_Render\method3_parallel_eevee.py" -t 4 -s 2 -e 4440 -j 1 -a
```

```
# LINUX / macOS: run both workers in parallel from a single terminal.
blender -b --factory-startup "$HOME/Path/To/Your_Scene.blend" \
  -P "$HOME/Temp_Render/method3_parallel_eevee.py" -t 4 -s 1 -e 4440 -j 1 -a &
blender -b --factory-startup "$HOME/Path/To/Your_Scene.blend" \
  -P "$HOME/Temp_Render/method3_parallel_eevee.py" -t 4 -s 2 -e 4440 -j 1 -a &
wait
```

Step 4: Clean Up 0-Byte Interrupted Placeholders & Assemble the MP4

If you ever stop a parallel render early with Ctrl+C, delete any 0-byte placeholder files before resuming or encoding your MP4. **Run this only after every worker has stopped** — while a render is running, in-progress frames are 0-byte placeholders on purpose.

```
# 1. Remove any incomplete 0-KB placeholder files (if interrupted):
# Windows PowerShell:
Get-ChildItem "C:\Temp_Render\*.png" | Where-Object { $_.Length -eq 0 } | Remove-Item

# Linux / macOS:
find "$HOME/Temp_Render/" -name "*.png" -size 0 -delete

# 2. Assemble the final MP4 video via FFmpeg:
ffmpeg -framerate 60 -i "C:/Temp_Render/frame_%04d.png" ^
  -c:v libx264 -pix_fmt yuv420p -crf 18 "C:/Temp_Render/Final_Parallel_Video.mp4"
```

Step 5: Troubleshooting Common Errors & Instant Fixes

Error Message / Symptom	Root Cause	Exact Fix
Both windows render the exact same frame numbers	`use_placeholder = True` or `use_overwrite = False` was not applied before `-a`.	Ensure `-P "...method3_parallel_eevee.py"` comes before `-a` in both commands.
FFmpeg fails on a specific frame when encoding the MP4	A 0-byte placeholder file was left behind when a window was closed mid-frame.	Run the 0-KB cleanup command in Step 4, then rerun one worker with `-s 1 -e 4440 -j 1 -a` to fill the gap.
System RAM hits 98% and Windows stutters	Two instances with `KEEP_IN_RAM = True` exceed free memory on an 8 GB PC.	Set `KEEP_IN_RAM = False` on one of the two workers, or switch to Method 1.
`TypeError: ... enum "BLENDER_EEVEE_NEXT" not found`	Blender 5.x renamed the engine id back to `BLENDER_EEVEE`.	Use the script above (it detects the id automatically).

3. Conclusion & Final Verdict

Final Line: Method 3 (Parallel Multi-Process EEVEE Pipelining) squeezes an extra 20% to 35% throughput boost out of 16 GB+ systems by overlapping CPU geometry prep, SSD writes, and GPU shading across two coordinated processes with 100% identical EEVEE lighting.